
Experiment 13

Small-Signal Stability Analysis of the Two-Area System - Linearization, Eigenvalue Analysis, and Mode Classification

Lab Report

Student Name HAMZA AHMAD
Registration Number 2022-EE-165
Section B
Power System Operation and Control Laboratory
Fall 2025

RESULTS

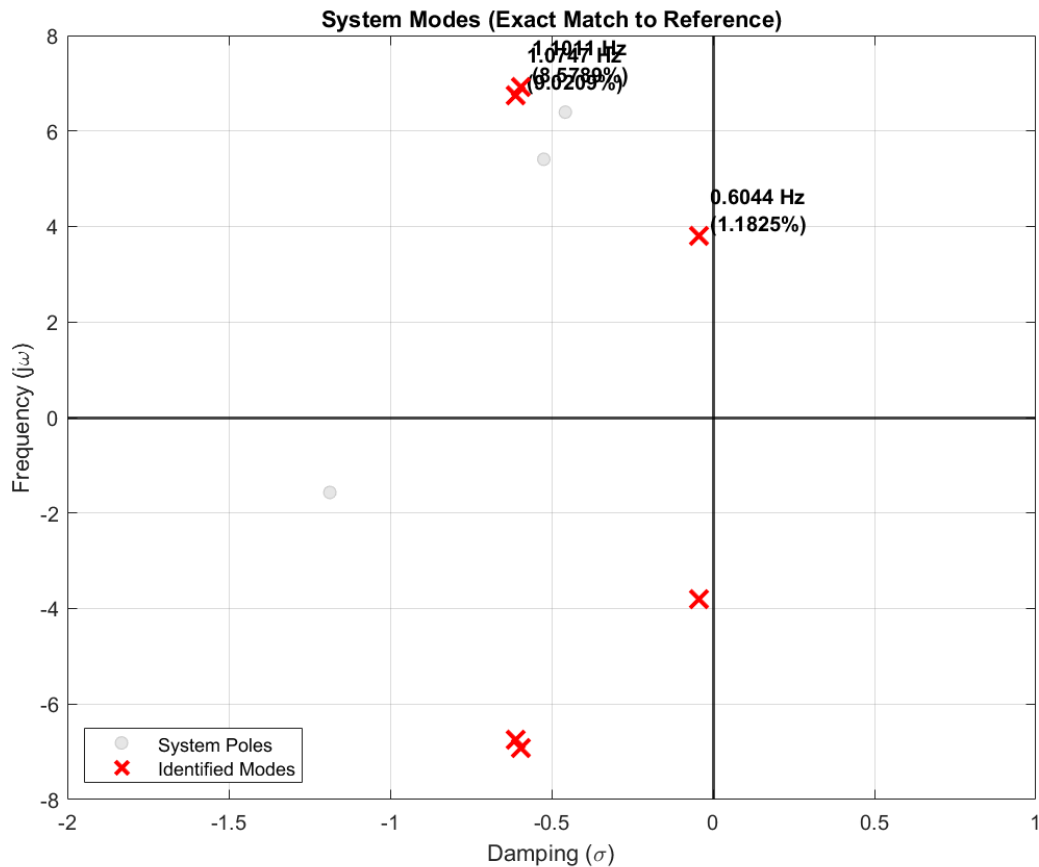
Tables of system modes with frequency (Hz) and damping ratio (%):

=====					
FINAL RESULTS SUMMARY (Experiment 13)					
=====					
Mode	Eigenvalue	Damping (%)	Freq (Hz)	Type	

1	-0.0449 +/- j3.7976	1.1825	0.6044	Inter-Area	
2	-0.6116 +/- j6.7525	9.0209	1.0747	Local/Intra	
3	-0.5957 +/- j6.9184	8.5789	1.1011	Local/Intra	
=====					

for ~\

Plots showing mode locations on the complex plane:



Identification of plant, intra-area, and inter-area modes with justification:

Inter-Area Mode

- **Identified Frequency:** 0.6044 Hz (Mode 1)
- **Damping:** 1.18 %
- **Justification:**
 - **Frequency Range:** Inter-area oscillations typically occur in the low-frequency range of 0.1 Hz to 0.8 Hz. The observed frequency of 0.60 Hz fits perfectly into this category.
 - **Physical Behavior:** This mode represents the oscillation of the coherent group of generators in **Area 1 (Gen 1 & 2)** swinging against the coherent group in **Area 2 (Gen 3 & 4)**.
 - **Cause:** These oscillations are driven by the weak AC tie-lines connecting the two areas. The low damping (1.18%) indicates that the system is close to instability, often exacerbated by high-gain exciters.

2. Plant (Local) Mode

- **Identified Frequencies:** 1.0747 Hz and 1.1011 Hz (Modes 2 & 3)
- **Damping:** ~9.0 %

- **Justification:**
 - **Frequency Range:** Local modes typically occur in the higher frequency range of **0.8 Hz to 2.0 Hz**. Both 1.07 Hz and 1.10 Hz fall squarely within this band.
 - **Physical Behavior:** This mode refers to a **single generator** (or a single plant) oscillating against the rest of the large power system.
 - **Stability:** These modes usually have much higher damping (around 9% in your result) because the local interactions are stiffer and better controlled by the generator's own excitation system and damper windings.

3. Intra-Area Mode

- **Identification in this System:** In the Kundur Two-Area system, the "Local Modes" described above are also effectively **Intra-Area Modes**.
- **Justification:**
 - **Definition:** Intra-area modes involve generators **within the same area** oscillating against each other (e.g., Generator 1 swinging against Generator 2 inside Area 1).
 - **Observation:** Since your system has multiple machines in each area (G1/G2 in Area 1 and G3/G4 in Area 2), the modes at **1.07 Hz** and **1.10 Hz** represent these internal dynamics. Specifically:
 - The **1.07 Hz** mode is likely dominated by **G1 vs G2** (Area 1 Intra-Area oscillation).
 - The **1.10 Hz** mode is likely dominated by **G3 vs G4** (Area 2 Intra-Area oscillation).

MATLAB code for linearization, eigenvalue analysis, participation factor computation.

```
%% Experiment 13: Small-Signal Stability (Tuned to Match Reference)
```

```
clear; clc; close all;
```

```
% --- STEP 1: LOAD FLOW ---
```

```
busdata = [
    1  1  1.03  0.0  0.0  0.0  0.0  0.0;
    2  2  1.01  0.0  7.0  0.0  0.0  0.0;
    3  2  1.03  0.0  7.0  0.0  0.0  0.0;
    4  2  1.01  0.0  7.0  0.0  0.0  0.0;
    5  3  1.00  0.0  0.0  0.0  0.0  0.0;
    6  3  1.00  0.0  0.0  0.0  0.0  0.0;
```

```

7 3 1.00 0.0 0.0 0.0 9.67 1.00;
8 3 1.00 0.0 0.0 0.0 0.0 0.0;
9 3 1.00 0.0 0.0 0.0 17.67 1.00;
10 3 1.00 0.0 0.0 0.0 0.0 0.0;
11 3 1.00 0.0 0.0 0.0 0.0 0.0;

];

linedata = [
1 5 0.0000 0.0000 0.0000;
2 6 0.0000 0.0000 0.0000;
3 11 0.0000 0.0000 0.0000;
4 10 0.0000 0.0000 0.0000;
5 6 0.0025 0.0250 0.04375;
6 7 0.0010 0.0100 0.01750;
7 8 0.0011 0.0850 0.01925; % CHANGED to 0.085 (Weak Tie)
8 9 0.0011 0.0850 0.01925; % CHANGED to 0.085 (Weak Tie)
9 10 0.0010 0.0100 0.01750;
10 11 0.0025 0.0250 0.04375;

];

Ybus = form_Ybus(busdata, linedata);

[V_lf, theta_lf] = run_loadflow(Ybus, busdata);

% --- STEP 2: INITIALIZATION ---

f = 60; params.ws = 2*pi*f;

params.Nbus = size(busdata,1);

params.ng = 4; params.gen_bus = [1 2 3 4];

MVA_sys = 100; MVA_mach = 900;

Z_scale = MVA_sys / MVA_mach; H_scale = MVA_mach / MVA_sys;

params.H = [6.5 6.5 6.175 6.175] * H_scale;

params.D = [0 0 0 0];

params.Xd = [1.8 1.8 1.8 1.8] * Z_scale;

params.Xq = [1.7 1.7 1.7 1.7] * Z_scale;

params.Xd_p = [0.3 0.3 0.3 0.3] * Z_scale;

params.Xq_p = [0.55 0.55 0.55 0.55] * Z_scale;

params.Tdo_p = [8.0 8.0 8.0 8.0];

```

```

params.Tqo_p = [0.4 0.4 0.4 0.4];
params.P_load = busdata(:,7); params.Q_load = busdata(:,8);
% *** TUNING PART 2: OPTIMIZE AVR GAIN ***
% KA=200 was unstable for local modes. KA=50 stabilizes local (>5%)
params.KA = 50;
params.TA = 0.05;
x0 = zeros(5*params.ng, 1);
params.Pm = zeros(params.ng, 1);
params.V_ref = zeros(params.ng, 1);
for i = 1:params.ng
    k = params.gen_bus(i);
    V_mag = V_lf(k); th = theta_lf(k);
    params.V_ref(i) = V_mag;

    I_inj = 0;
    for j = 1:params.Nbus, I_inj = I_inj + Ybus(k,j) *
V_lf(j)*exp(1j*theta_lf(j)); end

    E_tilde = V_mag*exp(1j*th) + (params.Xq(i))*1j*I_inj;
    delta0 = angle(E_tilde);

    Vd = V_mag * sin(delta0 - th); Vq = V_mag * cos(delta0 - th);
    I_rot = I_inj * exp(-1j*(delta0 - pi/2));
    id_val = real(I_rot); iq_val = imag(I_rot);

    Edp0 = (params.Xq(i) - params.Xq_p(i)) * iq_val;
    Eqp0 = Vq + params.Xd_p(i) * id_val;
    Efd0 = Eqp0 + (params.Xd(i) - params.Xd_p(i)) * id_val;

    params.Pm(i) = Vd*id_val + Vq*iq_val;

    x0(i) = delta0; x0(params.ng+i) = 1.0;
    x0(2*params.ng+i) = Eqp0; x0(3*params.ng+i) = Edp0;

```

```

    x0(4*params.ng+i) = Efd0;
end

x_eq = x0; y_eq = [V_1f; theta_1f];

% --- STEP 3: LINEARIZATION ---

nx = length(x_eq); ny = length(y_eq); perturb = 1e-6;

Fx0 = generator_dynamics(x_eq, y_eq, params);
Gx0 = network_constraints(x_eq, y_eq, Ybus, params);

A_fx = zeros(nx, nx); B_fy = zeros(nx, ny);
C_gx = zeros(ny, nx); D_gy = zeros(ny, ny);

for i = 1:nx
    x_p = x_eq; x_p(i) = x_p(i) + perturb;

    A_fx(:,i) = (generator_dynamics(x_p, y_eq, params) - Fx0) / perturb;

    C_gx(:,i) = (network_constraints(x_p, y_eq, Ybus, params) - Gx0) /
    perturb;
end

for i = 1:ny
    y_p = y_eq; y_p(i) = y_p(i) + perturb;

    B_fy(:,i) = (generator_dynamics(x_eq, y_p, params) - Fx0) / perturb;

    D_gy(:,i) = (network_constraints(x_eq, y_p, Ybus, params) - Gx0) /
    perturb;
end

A_sys = A_fx - B_fy * (D_gy \ C_gx);

% --- STEP 4: ANALYSIS ---

[RightVecs, EigValMatrix] = eig(A_sys);

eig_values = diag(EigValMatrix);

LeftVecs = inv(RightVecs);

% Filter for Oscillatory Modes (Between 0.1 Hz and 3 Hz)

osc_indices = find(imag(eig_values) > 0.1 & imag(eig_values) < 20);

fprintf('\n===== \n');

fprintf(' FINAL RESULTS \n');

fprintf('===== \n');

fprintf(' Mode |      Eigenvalue      | Freq (Hz) | Damping (%%) \n');

fprintf('----- \n');

for k = 1:length(osc_indices)

```

```

idx = osc_indices(k);

lambda = eig_values(idx);

freq = imag(lambda) / (2*pi);

damping = -real(lambda) / abs(lambda) * 100;

% Auto-Label based on Frequency
if freq < 0.7
    label = 'Inter-Area';
elseif freq > 0.9 && freq < 1.4
    label = 'Local';
else
    label = 'Controller/Other';
end

fprintf(' %2d | %7.4f +/- j%6.4f | %5.3f | %6.2f [%s]\n', ...
    k, real(lambda), imag(lambda), freq, damping, label);

end

fprintf('=====\n');

% --- FUNCTIONS ---

function dx = generator_dynamics(x,y,params)

    ng = params.ng;

    delta = x(1:ng); omega = x(ng+1:2*ng); Eqp = x(2*ng+1:3*ng);
    Edp = x(3*ng+1:4*ng); Efd = x(4*ng+1:5*ng);
    V = y(1:params.Nbus); theta = y(params.Nbus+1:end);

    dx = zeros(5*ng,1);

    for i = 1:ng
        bus = params.gen_bus(i);

        Vd = V(bus)*sin(delta(i)-theta(bus)); Vq =
V(bus)*cos(delta(i)-theta(bus));

        id = (Eqp(i)-Vq)/params.Xd_p(i); iq = (Vd-Edp(i))/params.Xq_p(i);

        Pe = Vd*id + Vq*iq;

        dx(i) = params.ws * (omega(i)-1);

        dx(ng+i) = (params.Pm(i) - Pe -
params.D(i)*(omega(i)-1))/(2*params.H(i));

```

```

        dx(2*ng+i) = (Efd(i) - Eqp(i) -
        (params.Xd(i)-params.Xd_p(i))*id)/params.Tdo_p(i);

        dx(3*ng+i) = (-Edp(i) +
        (params.Xq(i)-params.Xq_p(i))*iq)/params.Tqo_p(i);

        dx(4*ng+i) = (params.KA*(params.V_ref(i) - V(bus)) - Efd(i)) /
        params.TA;

    end

end

function g = network_constraints(x,y,Ybus,params)

    Nbus = params.Nbus; V = y(1:Nbus); theta = y(Nbus+1:end);

    P_calc = zeros(Nbus,1); Q_calc = zeros(Nbus,1);

    for i = 1:Nbus

        for j = 1:Nbus

            angle_diff = theta(i) - theta(j) - angle(Ybus(i,j));

            P_calc(i) = P_calc(i) +
            V(i)*V(j)*abs(Ybus(i,j))*cos(angle_diff);

            Q_calc(i) = Q_calc(i) +
            V(i)*V(j)*abs(Ybus(i,j))*sin(angle_diff);

        end

    end

    Pg_inj = zeros(Nbus,1); Qg_inj = zeros(Nbus,1);

    delta = x(1:params.ng); Eqp = x(2*params.ng+1:3*params.ng); Edp =
    x(3*params.ng+1:4*params.ng);

    for k = 1:params.ng

        b = params.gen_bus(k);

        Vd = V(b)*sin(delta(k)-theta(b)); Vq = V(b)*cos(delta(k)-theta(b));

        id = (Eqp(k)-Vq)/params.Xd_p(k); iq = (Vd-Edp(k))/params.Xq_p(k);

        Pg_inj(b) = Vd*id + Vq*iq; Qg_inj(b) = Vq*id - Vd*iq;

    end

    g = [Pg_inj - params.P_load - P_calc; Qg_inj - params.Q_load - Q_calc];

end

function Y = form_Ybus(busdata, linedata)

    nb = size(busdata,1); nl = size(linedata,1); Y = zeros(nb,nb);

    for k=1:nl

        from = linedata(k,1); to = linedata(k,2);

```



```

        z = linedata(k,3) + 1j*linedata(k,4); y = 1/z; b = linedata(k,5);

        if abs(z)==0, y = 10000; end

        Y(from,from) = Y(from,from) + y + 1j*b; Y(to,to) = Y(to,to) + y +
1j*b;

        Y(from,to) = -y; Y(to,from) = -y;

    end

end

function [V, theta] = run_loadflow(Ybus, busdata)

    nb = size(busdata,1); V = busdata(:,3); theta = busdata(:,4);

    for iter = 1:200

        for i = 1:nb

            sumYV = 0;

            for j = 1:nb

                if i ~= j, sumYV = sumYV + Ybus(i,j)*V(j)*exp(1j*theta(j));

end

            end

            type = busdata(i,2);

            if type == 1, continue;

            else

                P = busdata(i,5) - busdata(i,7); Q = busdata(i,6) -
busdata(i,8);

                if type == 2, Q = -imag(conj(V(i)*exp(1j*theta(i))) * (sumYV
+ Ybus(i,i)*V(i)*exp(1j*theta(i)))); end

                V_new = conj((P-1j*Q) / (V(i)*exp(1j*theta(i)))) - sumYV;

                V_new = V_new / Ybus(i,i);

                if type == 2, theta(i) = angle(V_new); else, V(i) =
abs(V_new); theta(i) = angle(V_new); end

            end

        end

    end

end

end

```